

EnergyFlow.jl: Optimal-transport distances between collider events in Julia

Matt LeBlanc^{1,2}[¶], Hanting Li¹, and Haochen Wang¹

¹ Department of Physics, Brown University, Providence, RI, USA ² The NSF AI Institute for Artificial Intelligence and Fundamental Interactions, Boston, MA, USA [¶] Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

The Energy Mover's Distance (EMD) ([Komiske et al., 2019](#)) provides a geometric way to compare collider events. An event is represented as a discrete distribution of energy or transverse momentum over a space of particle directions. The distance between two events is the minimum transport cost required to rearrange one distribution into the other, where the cost is the amount of energy moved multiplied by a chosen function of the angular distance. For normalized events, this construction is a discrete optimal-transport or Wasserstein distance ([Peyré & Cuturi, 2019](#)). This approach is inspired by the Earth Mover's Distance used in computer vision and image processing (?), and it generalizes the concept of event-shape observables to a continuous space of possible measurements.

Equipped with a metric, a sample of collider events forms a metric space, within which nearest-neighbor classification, clustering, dimensional reduction, and visualization are all well defined ([Komiske et al., 2019](#)). The EMD has been applied to jet substructure, event-shape observables, anomaly detection, and studies of the geometry of collider data ([Cesarotti & Thaler, 2020](#); [Komiske et al., 2020](#)). A single efficient EMD implementation, therefore, supports a wide range of collider physics applications.

EnergyFlow.jl is a Julia package for computing the EMD between collider events. It provides exact network-simplex solvers in Float64 and Float32, an approximate entropy-regularized Sinkhorn solver ([Cuturi, 2013](#)), and several ground metrics for collider and more general data. These include Euclidean distance, a periodic metric on the (y, ϕ) or (η, ϕ) plane, precomputed cost matrices, and user-defined metrics. The package supports single-pair and multithreaded pairwise calculations, together with workspace-reusing interfaces for repeated computations. Finally, it implements event isotropy ([Cesarotti & Thaler, 2020](#)), an event-shape observable that compares a collider event with a quasi-uniform reference distribution that has been measured at the LHC ([ATLAS Collaboration, 2023](#); [CMS Collaboration, 2025](#)) and calculated in perturbative quantum chromodynamics ([Atzori et al., 2026](#)).

Statement of need

Optimal-transport observables can be computationally demanding. The cost of each transport problem increases quickly with particle multiplicity, and applications requiring pairwise distance calculations scale quadratically with the number of events. Efficient solvers, memory reuse, and parallel execution are therefore important for applying the EMD to realistic collider datasets.

The established Python EnergyFlow package introduced widely used interfaces for collider EMD calculations ([Komiske et al., 2019](#)). Its transport calculations use compiled implementations provided by packages such as wasserstein and the Python Optimal Transport library (POT) ([Flamary et al., 2021](#)), which provide high-performance implementations and a mature

ecosystem. Researchers working primarily in Julia have lacked a native implementation that can be inspected, extended, and integrated directly with Julia analysis code. Modifying low-level solver behavior or introducing specialized distance calculations may otherwise require crossing language boundaries or developing additional compiled extensions.

EnergyFlow.jl fills a gap in the growing Julia high-energy-physics software ecosystem (Eschle & others, 2023) by providing native optimal-transport tools designed around collider-event data. It contains a native Julia implementation of the complete calculation, including the network-simplex solver. Julia provides high-level language features while compiling specialized numerical code to native instructions (Bezanson et al., 2017). As a result, solver implementation, ground-distance definitions, event processing, and user-facing analysis code can be developed within the same language and type system.

The network-simplex implementation is based on LEMON (Dezső et al., 2011). It uses a block-search pivot rule, an epsilon-scaled optimality criterion, succ_num-based least-common-ancestor calculations, and incremental updates of the spanning-tree representation. The :ot64 and :ot32 backends additionally use an arc-mixing strategy that improves pivot selection for degenerate or unbalanced transport problems; the benchmarks below show this mattering most for strongly asymmetric problems, where it is both faster and more robust than the plain block-search rule. A parallel block-search pivot rule following Kara and Özturan (Kara & Özturan, 2022) is also provided. It reaches the optimum in about a quarter fewer pivots than the serial rule across the range tested, but the per-pivot coordination cost means the saving only translates into faster wall-clock time for large single solves: it is about 1.35 times faster than the serial rule for events of 2000 to 3000 particles at moderate thread counts, and slower below roughly 1500 particles. It is selected on the solver workspace rather than through the user-facing calls.

For ensembles of events, the package parallelizes over independent event pairs using Julia tasks. A block-cyclic work assignment distributes consecutive groups of pairs among tasks. This scheduling strategy is intended to improve load balance when event multiplicity varies across the dataset. The package provides reusable workspaces that reduce memory allocation when many related transport problems are solved.

Verification and performance

The implementation is tested using problems with analytically known solutions and comparisons with independent optimal-transport software. The test suite compares the exact EnergyFlow.jl backends with the exact solver provided by POT (Flamary et al., 2021). Tests cover normalized and unnormalized event weights, multiple ground metrics, both floating-point precisions, pairwise result ordering, and transport-plan marginals. The event-isotropy implementation is also compared with the Python reference implementation on simulated collider events. The Sinkhorn backend is tested separately because it solves an entropy-regularized approximation and is not expected to reproduce the exact network-simplex result at finite regularization strength.

The repository includes reproducible benchmarks for single-pair, pairwise, and event-isotropy calculations, summarized in Figure 1. In the pairwise benchmark, the package computes 900 and 2500 event pairs drawn from a HepMC3 sample of simulated minimum-bias LHC collisions. On a single CPU thread, the Julia network-simplex backends achieve between 1.7 and 5.7 times the throughput of POT, depending on the ground metric, weight-normalization setting, and selected backend. The arc-mixing backend performs particularly well for unnormalized events with unequal total weights.

Pairwise calculations also benefit from task-level parallelism, distributing independent event pairs across Julia tasks. Wall time grows linearly with the number of pairs across the range measured, from five thousand to four million pairs. Throughput on 48 threads rises from about 7000 pairs per second on the smallest matrices, where fixed overhead and load imbalance

across uneven multiplicities are still visible, to a plateau near 10,000 pairs per second above roughly half a million pairs; a 2800-event distance matrix, just under four million pairs, takes about six and a half minutes. Against the multithreaded pairwise driver of the wasserstein library at the same thread count, which is the comparison at equal parallelism, EnergyFlow.jl is comparable at half a million pairs and about 1.1 times faster at four million.

In the event-isotropy benchmark, execution with eight Julia threads gives an end-to-end throughput between 8 and 43 times that of the single-threaded POT reference calculation across the ring, cylinder, and spherical reference geometries, with the largest margins on the finest spherical references. Across every geometry, backend, and thread count, the Float64 backends reproduce POT's mean isotropy to the six decimal places reported. These results are workload- and hardware-dependent and should not be interpreted as universal performance ratios.

The Sinkhorn backend is presently experimental and is not recommended in place of the exact solvers for the problem sizes considered here. Its values agree with POT's log-domain Sinkhorn to approximately nine significant figures at matched regularization strength, and it is several times faster than that implementation, but on a CPU it is orders of magnitude slower than the exact network simplex at collider multiplicities: reaching a relative accuracy of a few parts in a thousand for 500-particle events costs nearly four orders of magnitude more time than solving the problem exactly. The entropic bias also grows with multiplicity at fixed regularization strength, so larger events require smaller regularization and therefore more iterations. The algorithm's value lies in a different regime from the one measured here: its inner loop is dense and uniform across a batch of problems, which makes it a natural candidate for GPU batching, where many transport problems are solved simultaneously rather than one at a time. That is the direction in which the backend is expected to become useful, and it is retained as a basis for that work.

All Julia and Python benchmark scripts are included in the repository, along with the accuracy-versus-cost study of the Sinkhorn backend, the pivot-rule comparison, and the event-isotropy figure. The benchmark documentation records the processor model, operating system, Julia and Python versions, package versions, thread configuration, the commit and source directory of the package actually measured, input sample, and commands needed to reproduce the reported measurements.

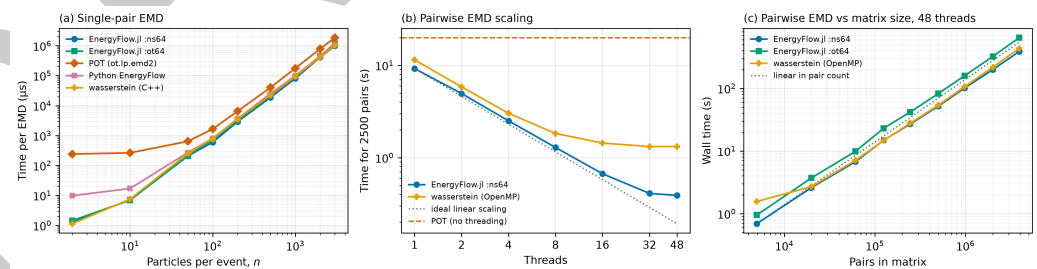


Figure 1: Performance of EnergyFlow.jl against Python optimal-transport implementations. (a) Wall time for a single EMD against particle multiplicity; the Julia backends avoid the per-call overhead visible in the Python interfaces at low multiplicity and track the same asymptotic behavior at high multiplicity. (b) Wall time for a 2500-pair distance matrix against Julia thread count, with the single-threaded POT time shown as a reference line and ideal linear speedup as a guide. (c) Wall time against the size of the distance matrix at fixed thread count, against linear growth in the number of pairs. All panels use the Euclidean ground metric with normalized event weights.

Example usage

Collider events are represented as matrices with one particle per row. The first column contains the particle weight—typically transverse momentum or energy—and the remaining columns

125 contain coordinates in the ground space. For example, events in the rapidity–azimuth plane
 126 use rows of the form (p_T, y, ϕ) .

sing EnergyFlow

```
events = load_hepmc3_events("events.hepmc"; maxevents=100)
```

```
# Compute the EMD between two events, treating azimuth as periodic.
```

```
distance = emd(  
    events[1],  
    events[2];  
    backend=:ns64,  
    metric=EtaPhiMetric(),  
    R=1.0,  
    beta=1.0,  
    norm=false,  
)
```

```
# Compute all  $n(n-1)/2$  distances among the loaded events.
```

```
distances = emds(  
    events;  
    backend=:ot64,  
    metric=EtaPhiMetric(),  
    R=1.0,  
    beta=1.0,  
    norm=false,  
)
```

127 The first call computes one EMD using the Float64 network-simplex backend. The second
 128 computes the strict upper triangle of the pairwise distance matrix and returns it as a vector in
 129 SciPy pdist order. Pairwise calculations are distributed across the available Julia threads.

130 For repeated single-pair calculations, a reusable workspace reduces memory allocation:

```
max_particles = maximum(size(event, 1) for event in events)
```

```
workspace = EMDWorkspace(  
    max_particles,  
    max_particles;  
    R=1.0,  
    beta=1.0,  
    norm=false,  
    metric=EtaPhiMetric(),  
)
```

```
distance = emd!(  
    workspace,  
    events[1],  
    events[2];  
    backend=:ns64,  
)
```

131 Julia must be started with multiple threads, for example `julia --threads=auto`, for the pairwise
 132 calculation to use more than one thread.

133 Availability and documentation

134 EnergyFlow.jl is released under the MIT license and is developed openly at [https://github.com/leblanc-](https://github.com/leblanc-lab/EnergyFlow.jl)
 135 lab/EnergyFlow.jl. The documentation is available at <https://leblanc-lab.github.io/EnergyFlow.jl/stable>
 136 and includes installation instructions, a getting-started guide, a tutorial using collider events,
 137 descriptions of the available solver backends and ground metrics, and a complete API reference.

138 The repository also includes runnable examples of single-pair and pairwise EMD calculations
 139 and event-isotropy calculations, provided as scripts, notebooks, and step-by-step walkthroughs.

140 Acknowledgements

141 This material is based on work supported by the U.S. Department of Energy, Office of Science,
 142 Office of High Energy Physics under Award Number DE-SC0026285.

143 This work is supported by the National Science Foundation under Cooperative Agreement
 144 PHY-2019786 (The NSF AI Institute for Artificial Intelligence and Fundamental Interactions,
 145 <http://iaifi.org/>).

146 This research was conducted using computational resources and services at the Center for
 147 Computation and Visualization, Brown University; it also received support from the Brown
 148 University Undergraduate Teaching and Research Awards (UTRA) program and from a Brown
 149 University Data Science Institute seed grant.

150 References

151 ATLAS Collaboration. (2023). Measurements of multijet event isotropies using optimal
 152 transport with the ATLAS detector. *JHEP*, 10, 060. [https://doi.org/10.1007/](https://doi.org/10.1007/JHEP10(2023)060)
 153 [JHEP10\(2023\)060](https://doi.org/10.1007/JHEP10(2023)060)

154 Atzori, D., Cacciari, M., Marzani, S., & Soyez, G. (2026). *Event isotropy in perturbative QCD*.
 155 <https://arxiv.org/abs/2606.23924>

156 Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A Fresh Approach to
 157 Numerical Computing. *SIAM Review*, 59(1), 65–98. <https://doi.org/10.1137/141000671>

158 Cesarotti, C., & Thaler, J. (2020). A Robust Measure of Event Isotropy at Colliders. *JHEP*,
 159 08, 084. [https://doi.org/10.1007/JHEP08\(2020\)084](https://doi.org/10.1007/JHEP08(2020)084)

160 CMS Collaboration. (2025). Measurement of event shapes in minimum-bias events from
 161 proton-proton collisions at $\sqrt{s}=13$ TeV. *Phys. Rev. D*, 112(11), 112006. [https://doi.org/10.](https://doi.org/10.1103/rqvb-gfb7)
 162 [1103/rqvb-gfb7](https://doi.org/10.1103/rqvb-gfb7)

163 Cuturi, M. (2013). Sinkhorn Distances: Lightspeed Computation of Optimal Transport.
 164 *Advances in Neural Information Processing Systems*, 26, 2292–2300.

165 Dezső, B., Jüttner, A., & Kovács, P. (2011). LEMON – an Open Source C++ Graph
 166 Template Library. *Electronic Notes in Theoretical Computer Science*, 264(5), 23–45.
 167 <https://doi.org/10.1016/j.entcs.2011.06.003>

168 Eschle, J., & others. (2023). Potential of the Julia Programming Language for High Energy
 169 Physics Computing. *Comput. Softw. Big Sci.*, 7(1), 10. [https://doi.org/10.1007/s41781-](https://doi.org/10.1007/s41781-023-00104-x)
 170 [023-00104-x](https://doi.org/10.1007/s41781-023-00104-x)

171 Flamary, R., Courty, N., Gramfort, A., Alaya, M. Z., Boissunon, A., Chambon, S., Chapel,
 172 L., Corenflos, A., Fatras, K., Fournier, N., Gautheron, L., Gayraud, N. T. H., Janati, H.,
 173 Rakotomamonjy, A., Redko, I., Rolet, A., Schutz, A., Seguy, V., Sutherland, D. J., ...
 174 Vayer, T. (2021). POT: Python Optimal Transport. *Journal of Machine Learning Research*,
 175 22(78), 1–8. <http://jmlr.org/papers/v22/20-451.html>

- 176 Kara, G., & Özturan, C. (2022). Parallel network simplex algorithm for the minimum cost
177 flow problem. *Concurrency and Computation: Practice and Experience*, 34(4), e6659.
178 <https://doi.org/10.1002/cpe.6659>
- 179 Komiske, P. T., Metodiev, E. M., & Thaler, J. (2019). Metric Space of Collider Events. *Phys.*
180 *Rev. Lett.*, 123(4), 041801. <https://doi.org/10.1103/PhysRevLett.123.041801>
- 181 Komiske, P. T., Metodiev, E. M., & Thaler, J. (2020). The Hidden Geometry of Particle
182 Collisions. *JHEP*, 07, 006. [https://doi.org/10.1007/JHEP07\(2020\)006](https://doi.org/10.1007/JHEP07(2020)006)
- 183 Peyré, G., & Cuturi, M. (2019). Computational Optimal Transport. *Foundations and Trends*
184 *in Machine Learning*, 11(5-6), 355–607. <https://doi.org/10.1561/22000000073>

DRAFT